

第八章 排序

一. 排序的基本概念

1. 排序: 重新排列表中的元素, 使表中的元素满足按关键字有序的过程。

2. 评价指标

(1) 稳定性: 关键字相同的元素经过排序后相对顺序是否会改变。

例:

6 3 2 5 1 4 3
↓ 稳定的 ↓ 不稳定的
1 2 3 3 4 5 6 1 2 3 3 4 5 6

(2) 时间复杂度、空间复杂度

3. 分类

(1) 内部排序: 指在排序期间元素全部存放在内存中的排序。

(2) 外部排序: 指在排序期间元素无法全部同时存放在内存中, 必须根据要求不断地在内、外存之间移动的排序。

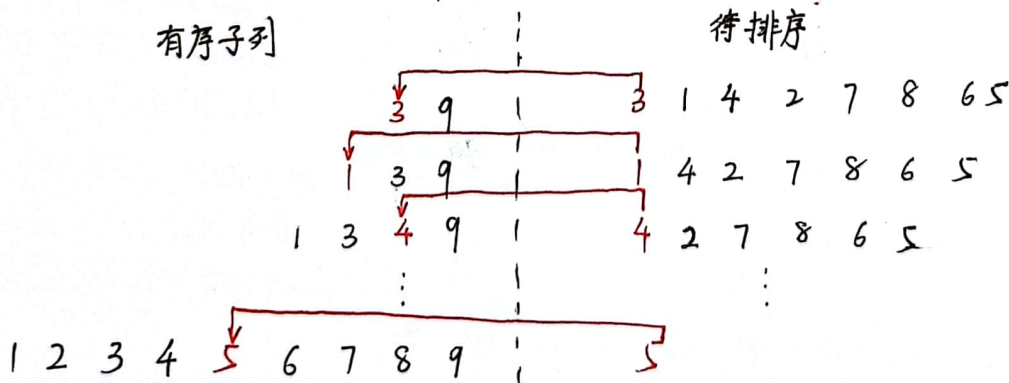
多路归并排序

插入 { 直接插入
折半插入
希尔
冒泡
交换 { 快速
简单选择
基数
归并
选择

二. 插入排序

1. 直接插入排序

(1) 算法思想: 每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中, 直到全部记录插入完成。



(2) 代码: void InsertSort (ElemType A[], int n) {

int i, j;

for (i = 2; i <= n; i++)

if (A[i] < A[i-1]) {

A[0] = A[i];

for (j = i-1; A[0] < A[j]; --j)

A[j+1] = A[j];

A[j+1] = A[0];

// 依次将 A[i] ~ A[n] 插入前面已排序序列

// 若 A[i] 小于前驱, 将 A[i] 插入有序表

// 复制为哨兵, A[0] 不存放任何元素

// 从后往前找待插入位置

// 向后挪位

// 复制到插入位置

(3) 算法性能分析

① 空间复杂度 $O(1)$: 仅使用了常数个辅助单元。

② 时间复杂度 $O(n^2)$: 最好情况: 已有序, 比较 $n-1$ 次, 不用移动 $O(n)$

总趟数 $n-1$ 最坏情况: 全逆序, 比较 $\frac{(n-1)(n+2)}{2}$ 次, 移动 $\frac{(n-1)(n+4)}{2}$ 次 $O(n^2)$

③ 稳定

无须移动元素

④ 适用性: 顺序存储和链式存储的线性表。

2. 折半插入排序

(1) 算法思想: 先用折半查找找到应该插入的位置, 再移动元素。

(2) 代码: `void InsertSort (ElemType A[], int n) {`

`int i, j, low, high, mid;`

`for (i=2; i<=n; i++) {`

`A[0] = A[i];`

`low = 1; high = i-1;`

`while (low <= high) {`

`mid = (low + high) / 2;`

`if (A[mid] > A[0]) high = mid - 1;`

`else low = mid + 1;`

`}`

`for (j=i-1; j>=high+1; --j)`

`A[j+1] = A[j];`

`A[high+1] = A[0];`

`}`

`}`

// 设置折半查找的范围

// 折半查找 (默认递增有序)

// 查左半子表

// 查右半子表

// 向后挪位

// 插入元素

总趟数: $n-1$

仅减少了比较的次数 $O(n \log n)$

10	20	30	40	55	60	60	70	80	90	10
	↑			↑					↑	↑
	low			mid					high	i

↓

10	20	30	40	55	60	60	70	80	90	10
↑	↑									↑
high	low									i

↓

10	10	20	30	40	55	60	60	70	80	90
----	----	----	----	----	----	----	----	----	----	----

注意:

① 当 $low > high$ 时折半查找停止, 应将 $[low, i-1]$ 内的元素全部右移, 并将 $A[0]$ 复制到 low 所指位置。

② 当 $A[mid] == A[0]$ 时, 为保证算法“稳定性”, 应继续在 mid 所指位置右边寻找插入位置。

(3) 算法性能

① 空间复杂度 $O(1)$

② 时间复杂度 $O(n^2)$

③ 稳定

④ 适用性: 仅顺序存储的线性表。

3. 希尔排序

(1) 算法思想：先将待排序表分割成若干形如 $[i, i+d, i+2d, \dots, i+kd]$ 的“特殊子表”，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直至 $d=1$ 。

待排序表：

49	38	65	97	76	13	27	49
----	----	----	----	----	----	----	----

[	49	—————	76	]
	38	—————	13	
	65	—————	27	
	97	—————	49	

第一趟结果：
 $(d_1 = \frac{n}{2} = 4)$

49	13	27	49	76	38	65	97
----	----	----	----	----	----	----	----

[	49	—————	27	—————	76	—————	65	]
	13	—————	49	—————	38	—————	97	

第二趟结果：
 $(d_2 = \frac{d_1}{2} = 2)$

27	13	49	38	65	49	76	97
----	----	----	----	----	----	----	----

第三趟结果：
 $(d_3 = \frac{d_2}{2} = 1)$

13	27	38	49	49	65	76	97
----	----	----	----	----	----	----	----

(2) 代码：

```
void ShellSort (ElemType A[], int n) {
    int d, i, j;    // A[0] 只是暂存单元，不是哨兵，当 j<=0 时，插入位置已到
    for (d = n/2; d >= 1; d = d/2)    // 增量变化
        for (i = d+1; i <= n; ++i)
            if (A[i] < A[i-d]) {    // 将 A[i] 插入有序增量子表
                A[0] = A[i];    // 暂存在 A[0]
                for (j = i-d; j > 0 && A[0] < A[j]; j -= d)
                    A[j+d] = A[j];    // 记录后移
                A[j+d] = A[0];    // 插入
            }
}
```

(3) 算法性能

① 空间复杂度 $O(1)$ 。

② 时间复杂度 $O(n^3)$ ，最坏情况下 $O(n^3)$

③ 不稳定

④ 适用性：仅顺序表

三. 交换排序

1. 冒泡排序

(1) 算法思想: 从后往前 (或从前往后) 两两比较相邻元素的值, 若为逆序 ($A[j-1] > A[j]$), 则交换它们, 直到序列比较完。这样的过程为一趟冒泡排序。

(2) 代码: `void BubbleSort (ElemType A[], int n) {`

`for (int i=0; i<n-1; i++) {`

`bool flag == false;`

// 表示本趟冒泡是否发生交换的标志

`for (int j=n-1; j>i; j--)`

// 一趟冒泡排序过程, 从后往前

`if (A[j-1] > A[j]) {`

// 若为逆序

`swap (A[j-1], A[j]);`

// 交换

`flag = true;`

`}`

`if (flag == false)`

// 本趟遍历没有发生交换, 说明表已经有序

`return;`

`}`

初始状态: 49 38 65 97 76 13 27 49

第一趟后: 13 49 38 65 97 76 27 49

从后往前

使关键字值最小的元素“冒”到最前面

第二趟后: 13 27 49 38 65 97 76 49

每趟排序都会将一个元素放置到其最终的位置上。

第三趟后: 13 27 38 49 49 65 97 76

已经确定最终位置的元素不用再对比

第四趟后: 13 27 38 49 49 65 76 97

:

最终状态: 13 27 38 49 49 65 76 97

(3) 算法性能

① 空间复杂度 $O(1)$

② 时间复杂度 $O(n^2)$: 最好情况: 有序, 比较 $n-1$ 次, 交换 0 次, $O(n)$

最坏情况: 逆序, 比较 $\frac{(n-1)n}{2}$ 次, 交换 $\frac{(n-1)n}{2}$ 次, $O(n^2)$

③ 稳定

④ 适用性: 顺序表、链表。

`void swap (int &a, int &b) {`

`int temp = a;`

`a = b;`

`b = temp;`

`}`

每次交换移动
3次元素

2. 快速排序

(1) 算法思想: ① 基于分治法

② 在待排序表中任取一个元素 pivot 为基准,

③ 通过一趟排序将待排序表划分成独立的两部分 $\left\{ \begin{array}{l} L[1 \dots k-1] \text{ 元素} < \text{pivot} \\ L[k+1 \dots n] \text{ 元素} \geq \text{pivot} \end{array} \right.$

④ pivot 放在 $L(k)$ 上, 上述过程为一次划分,

⑤ 对两个子表递归上述过程, 直至每部分内只有一个元素或为空为止。

(2) 代码:

```
void QuickSort(ElemType A[], int low, int high) {
    if (low < high) { // 递归跳出条件
        int pivotpos = Partition(A, low, high); // 划分
        QuickSort(A, low, pivotpos - 1); // 依次对两个子表进行递归排序
        QuickSort(A, pivotpos + 1, high);
    }
}
```

```
int Partition(ElemType A[], int low, int high) { // 一趟划分
    ElemType pivot = A[low]; // 将表首元素设为基准
    while (low < high) {
        while (low < high && A[high] >= pivot) -- high;
        A[low] = A[high]; // 找到比基准小的元素, 移到左端
        while (low < high && A[low] <= pivot) ++ low;
        A[high] = A[low]; // 找到比基准大的元素, 移到右端
    }
    A[low] = pivot; // 基准元素放在最终位置
    return low; // 返回存放基准元素的位置
}
```

待排序表: 49 38 65 97 76 13 27 49
 更小的元素交换到右边 ↑
 更大的元素交换到左边 ↓
 low → ← high

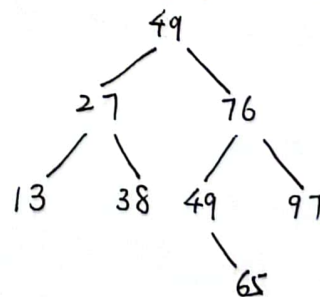
第一趟后: {27 38 13} 49 {76 97 65 49}

第二趟后: {13} 27 {38} 49 {49 65} 76 {97}

第三趟后: 13 27 38 49 49 {65} 76 97

第四趟后: 13 27 38 49 49 65 76 97

基准元素最终位置先确定



(3) 算法性能 (所有内部排序算法中平均性能最优)

① 空间复杂度 $O(\log_2 n)$: 与递归调用的层数大致 $\left\{ \begin{array}{l} \text{最好情况, } O(\log_2 n) \\ \text{最坏情况, } n-1 \text{ 次递归, } O(n) \end{array} \right.$

② 时间复杂度 $O(n \log_2 n)$: 最好情况, $O(n \log_2 n)$. 每次递归的基准都能将序列划分成均匀的两部分

最坏情况, 有序或逆序, $O(n^2)$

③ 不稳定

每次选择的是最靠边的元素

④ 适用性: 仅顺序表。

四. 选择排序

1. 简单选择排序

(1) 算法思想: 每一趟在待排序元素中选取关键字最小的元素加入有序子序列

(2) 代码:

```
void SelectSort (ElemType A[], int n) {  
    for (int i=0; i<n-1; i++) {  
        int min=i; // 记录最小元素位置  
        for (int j=i+1; j<n; j++) // 选择最小的元素  
            if (A[j]<A[min]) min=j; // 更新最小元素的位置  
        if (min!=i) swap(A[i], A[min]);  
    }  
}
```

待排序表: 49 38 65 97 76 13 27 49

第一趟后: 13 49 38 65 97 76 27 49

第二趟后: 13 27 49 38 65 97 76 49

:

第七趟后: 13 27 38 49 49 65 76 97 → 最后剩一个不用处理

(3) 算法性能

① 空间复杂度 $O(1)$

② 时间复杂度 $O(n^2)$: 最好情况, 移动 0 次; 最坏情况, 移动 $3(n-1)$ 次。

元素间的比较次数始终为 $\frac{n(n-1)}{2}$ 次。

③ 不稳定

④ 适用性: 顺序表, 链表, 关键字较少的情况。

2. 堆排序

(1) 算法思想

① 堆: 若 n 个关键字序列 $L[1 \dots n]$ 满足下面某一条性质, 则称为堆 (Heap):

大根堆 (大顶堆): $L(i) \geq L(2i)$ 且 $L(i) \geq L(2i+1)$, $1 \leq i \leq \lfloor n/2 \rfloor$

小根堆 (小顶堆): $L(i) \leq L(2i)$ 且 $L(i) \leq L(2i+1)$, $1 \leq i \leq \lfloor n/2 \rfloor$

② 思路: 将无序序列构建成一个堆, 根据升序/降序选择大/小顶堆;

将堆顶元素与末尾元素交换, 将最大/小元素“沉”到数组末端;

重新调整结构, 使其满足堆定义;

直到堆中仅剩一个元素为止。

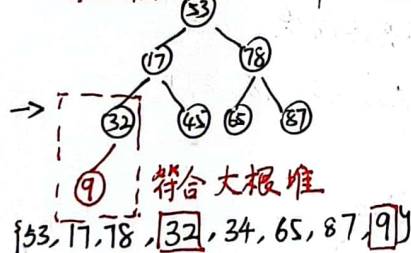
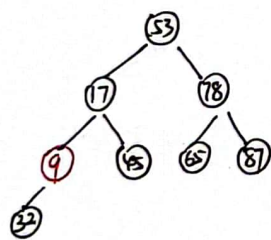
(2) 代码: 把所有非终端结点都检查一遍; 根 = 左、右

① 建立大根堆: 若不满足, 将当前结点与更大的一个孩子互换

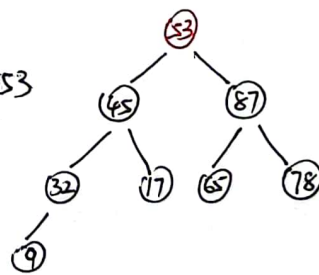
```
void BuildMaxHeap (ElemType A[], int len) {  
    for (int i = len/2; i > 0; i--) // 从 i = [n/2] ~ 1, 反复调整堆  
        HeadAdjust (A, i, len);  
}  
  
void HeadAdjust (ElemType A[], int k, int len) {  
    A[0] = A[k]; // A[0] 暂存子树的根结点  
    for (int i = 2*k; i <= len; i *= 2) // 沿 key 较大的子结点向下筛选  
        if (i < len && A[i] < A[i+1]) // 取 key 较大的子结点的下标  
            i++;  
        if (A[0] >= A[i]) break; // 筛选结束  
        else {  
            A[k] = A[i]; // 将 A[i] 调整到双亲结点上  
            k = i; // 修改 k 值, 以便继续向下筛选  
        }  
    }  
    A[k] = A[0]; // 被筛选结点的值放入最终位置  
}
```

初始序列: { 53, 17, 78, 9, 45, 65, 87, 32 }

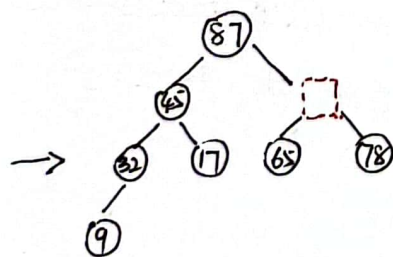
从最底层的分支结点开始调整



依次调整至结点 53



{ 53, 45, 87, 32, 17, 65, 78, 9 }
↑ k ↑ i ↑ i+1 右孩子更大 A[i] = 87



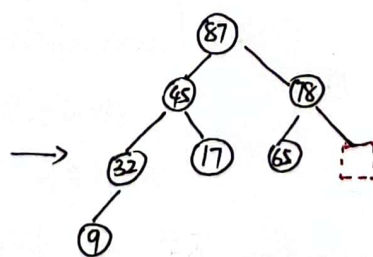
{ 87, 45, 78, 32, 17, 65, 9 }

↑ k

{ 87, 45, 78, 32, 17, 65, 9 }

↑ k

右孩子更大 A[i] = 78



{ 87, 45, 78, 32, 17, 65, 9 }

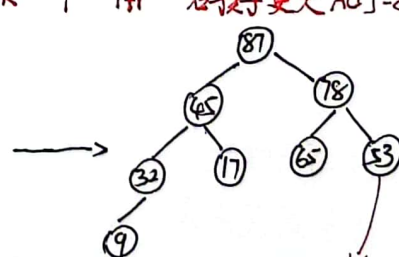
↑ k

{ 87, 45, 78, 32, 17, 65, 9 }

↑ k

大根堆

{ 87, 45, 78, 32, 17, 65, 53, 9 }

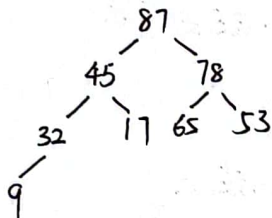


小元素不断“下坠”

② 基于大根堆进行排序:

```
void HeapSort (ElemType A[], int len) {
    BuildMaxHeap (A, len);           // 初始建堆
    for (int i = len; i > 1; i--) {   // n-1 趟的交换和建堆过程
        Swap (A[i], A[1]);           // 输出堆顶元素
        HeadAdjust (A, 1, i-1);      // 把剩余的 i-1 个元素整理成堆
    }
}
```

大根堆: {87, 45, 78, 32, 17, 65, 53, 9}

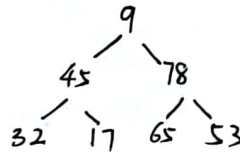


第1趟

{9, 45, 78, 32, 17, 65, 53, 87}

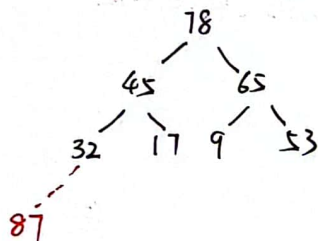
输出87

每一趟将堆顶元素加入有序子序列, 与最后一个元素交换 87



{78, 45, 65, 32, 17, 9, 53, 87}

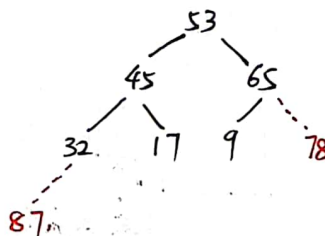
调整至大根堆



第2趟

输出78

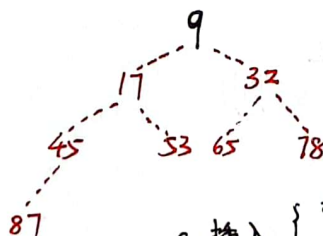
{53, 45, 65, 32, 17, 9, 78, 87}



只剩最后一个元素, 不用再调整

{9, 17, 32, 45, 53, 65, 78, 87}

调整... 第7趟后
输出



基于大根堆的堆排序得到的是升序序列。

(3) 算法性能

① 空间复杂度 $O(1)$

② 时间复杂度 $O(n \log_2 n)$: 下方有两个孩子, 下坠一层, 对比关键字 2 次,

建堆 $O(n)$ 下方有一个孩子, 下坠一层, 对比关键字 1 次,

③ 不稳定

若树高 h , 结点在 i 层, 最多下坠 $h-i$ 层, 关键字对比 $\leq 2(h-i)$ 次,
+ $h = \lfloor \log_2 n \rfloor + 1, \sum_{i=h-1}^1 2^{i-1} \times 2(h-i) \leq 4n$.

④ 适用性: 仅顺序表

堆 { 插入 { 新元素放在表尾(堆底) 每次
根据堆性质, 新元素不断“上升” (只需对比关键字 1 次)
删除 { 被删除元素用表尾(堆底)元素替代 (对比 1 次/2 次)
根据堆性质, 替代元素不断“下坠”
排序 { 每一趟交换后, 都需将根结点下坠调整,
 $O(n \log_2 n)$ 每一趟排序复杂度不超过 $O(h) = O(\log_2 n)$,
共 $n-1$ 趟, $O(n \log_2 n)$

五. 归并排序

1. 算法思想: 将两个或两个以上的有序表合并成一个新的有序表

$\begin{cases} 2 \text{路归并: 二合一} \\ k \text{路归并: } k \text{合一} \end{cases}$

① 若 $low < high$, 则将序列从中间 $mid = (low + high) / 2$ 分开。

② 左半部分 $[low, mid]$, 右半部分 $[mid+1, high]$ 分别递归地归并排序。

③ 左右两部分 Merge 为一个。

2. 代码:

```
ElemType *B = (ElemType*) malloc((n+1) * sizeof(ElemType)); // 辅助数组 B
void Merge (ElemType A[], int low, int mid, int high) { // 左半、右半表各自有序, 并成一个
    int i, j, k;
    for (k = low; k <= high; k++)
        B[k] = A[k]; // 将 A 中所有元素复制到 B 中
    for (i = low, j = mid+1, k = i; i <= mid && j <= high; k++) {
        if (B[i] <= B[j]) // 比较 B 的两个段中的元素
            A[k] = B[i++]; // 将较小值复制到 A 中
        else
            A[k] = B[j++];
    }
    while (i <= mid) A[k++] = B[i++]; // 若左半表未检测完, 复制到 A
    while (j <= high) A[k++] = B[j++]; // 若右半表未检测完, 复制到 B
}

void MergeSort (ElemType A[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2; // 从中间划分两个子序列
        MergeSort (A, low, mid); // 对左半序列递归排序
        MergeSort (A, mid+1, high); // 对右半序列递归排序
        Merge (A, low, mid, high); // 归并
    }
}
```

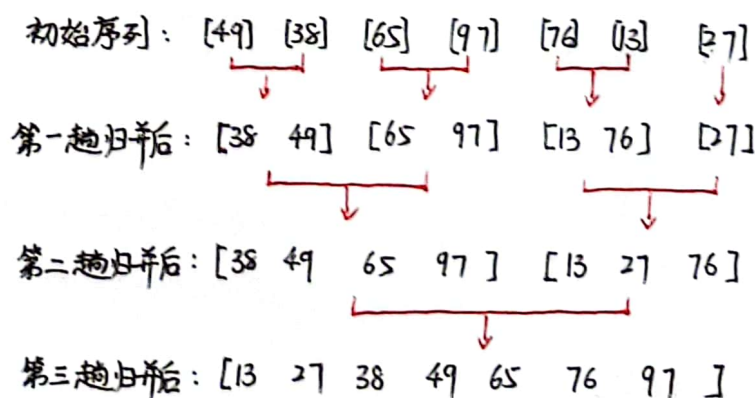
3. 算法性能

(1) 空间复杂度 $O(n)$: 需要 n 个辅助单元

(2) 时间复杂度 $O(n \log_2 n)$: 2 路归并, 每趟 $O(n)$,
归并趟数 $\lceil \log_2 n \rceil$ 。

(3) 稳定

(4) 适用性: 顺序表、链表



倒立的二叉树

$n \leq 2^{h-1}$, 即 $h-1 = \lceil \log_2 n \rceil$

六. 基数排序

1. 算法思想: 基于关键字各位的大小进行排序

- 最高位优先(MSD): 按位权重递减逐层划分
- 最低位优先(LSD): 按位权重递增逐层划分

最高位优先(MSD): 按位权重递减逐层划分

最低位优先(LSD):按位权重递增逐层划分

- ① 将关键字拆分为 d 位 (组)。
- ② 按照各个关键字位权重递增的次序 (如: 个、十、百), 做 d 趟 “分配” 和 “收集”, 若当前处理的关键字位可能取得 r 个值, 则需要建立 r 个队列。
- ③ 分配: 顺序扫描各个元素, 根据当前处理的关键字位, 将元素插入相应^队队列。
- ④ 收集: 把各个队列中的结点依次出队并链接。

2. 过程分析

初始序列: 278 $\rightarrow$ 109 $\rightarrow$ 063 $\rightarrow$ 930 $\rightarrow$ 589 $\rightarrow$ 184 $\rightarrow$ 505 $\rightarrow$ 269 $\rightarrow$ 008 $\rightarrow$ 083

基数 $r=10$ ，需要借助 10 个链队列。

每个关键字由3位子关键字构成 —— $k_1 k_2 k_3$ (百位, 十位, 个位)

→ LSD方式

第一趟分配后：Q[0] Q[1] Q[2] Q[3] Q[4] Q[5] Q[6] Q[7] Q[8] Q[9]

(对K3)

↓ ↓ ↓ ↓ ↓

930 063 184 505 278 109

↓ ↓ ↓ ↓ ↓

083 008 589 269

第一趟收集后: 930 $\rightarrow$ 063 $\rightarrow$ 083 $\rightarrow$ 184 $\rightarrow$ 505 $\rightarrow$ 278 $\rightarrow$ 008 $\rightarrow$ 109 $\rightarrow$ 589 $\rightarrow$ 267

第二趟分配后：

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]	Q[5]	Q[6]	Q[7]	Q[8]	Q[9]
↓			↓			↓	↓	↓	
505			930			063	278	083	
↓						↓		↓	
008						269		184	
↓								↓	
109								589	

第二趟收集后: 505 $\rightarrow$ 008 $\rightarrow$ 109 $\rightarrow$ 930 $\rightarrow$ 063 $\rightarrow$ 269 $\rightarrow$ 278 $\rightarrow$ 083 $\rightarrow$ 184 $\rightarrow$ 589

第三趟...后: 008 $\rightarrow$ 063 $\rightarrow$ 083 $\rightarrow$ 109 $\rightarrow$ 184 $\rightarrow$ 269 $\rightarrow$ 278 $\rightarrow$ 505 $\rightarrow$ 589 $\rightarrow$ 930 递增序列
(对 k_1)

3. 算法性能

- (2) 时间复杂度 $O(dn+r)$: 一趟收集合并 r 个队列, $O(r)$
一趟分配遍历所有关键字, $O(n)$

(3) 稳定

(4) 适用性：顺序表和链表。

① 数据元素的关键字可以方便地拆分为 d 组, 且 d 较小.
② 每组关键字的取值范围不大, 即 r 较小.
③ 数据元素个数 n 较大

七. 外部排序

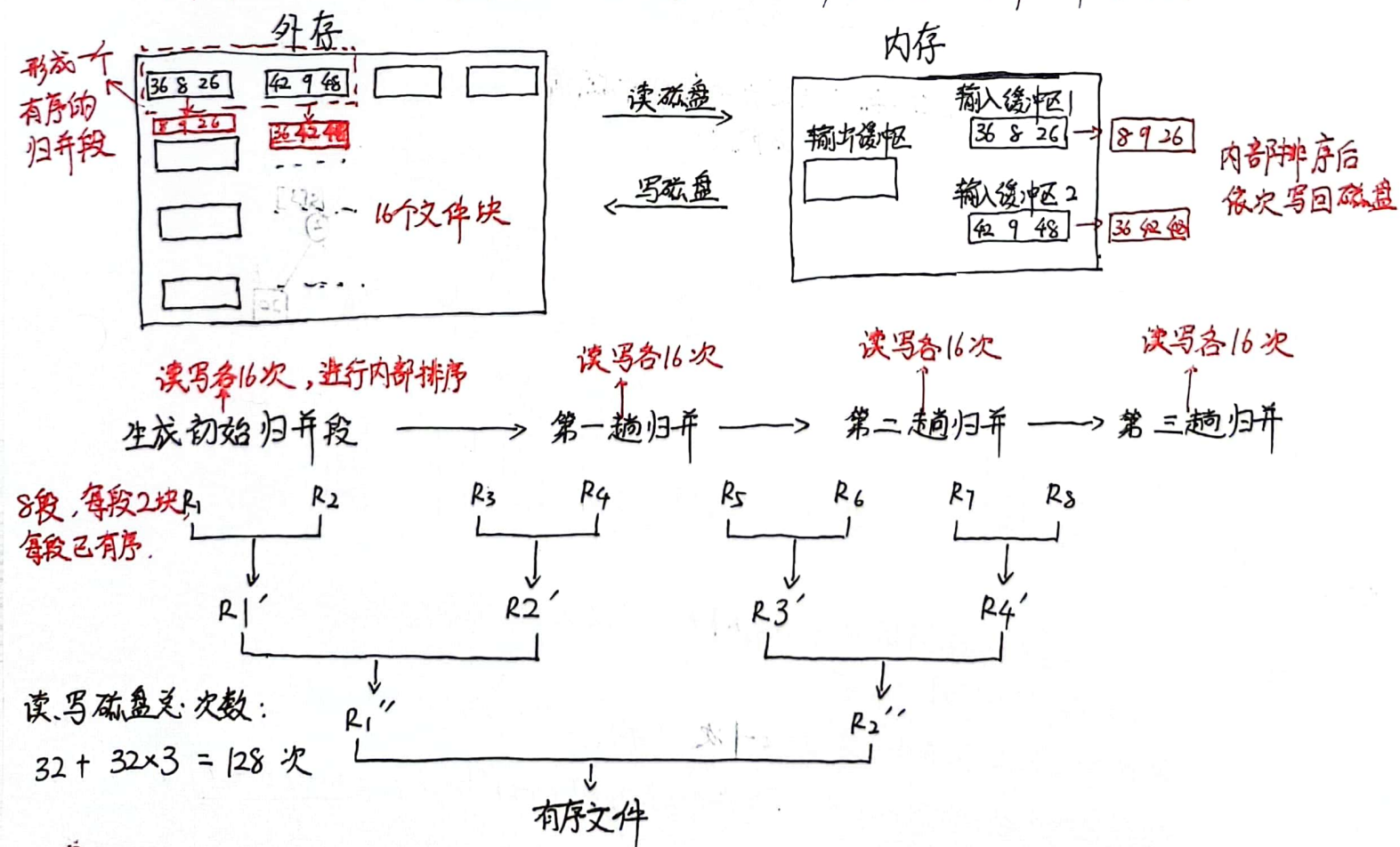
1. 基本概念

- ① 外部排序指的是大文件的排序，即待排序的记录存储在外存中，待排序的文件无法一次性装入内存，需要在内存和外存之间进行多次数据交换，以达到排序整个文件的目的。
- ② 为减少平衡归并中外存读/写次数所采取的方法：增大归并路数和减少归并段个数。
- ③ 利用败者树增大归并路数。
- ④ 利用置换-选择排序增大归并段长度来减少归并段个数。
- ⑤ 由长度不等的归并段进行多路平衡归并，需要构造最佳归并树。

2. 基本排序方法 (通常采用归并排序)

- (1) 算法步骤：
 - ① 根据内存缓冲区大小，将外存上的文件分成若干长度 l 的子文件，依次读入内存并利用内部排序算法排序，得到有序子文件后重新写回外存
 - ② 对这些归并段进行逐趟归并，使归并段逐渐由小到大，直至得到整个有序文件为止。

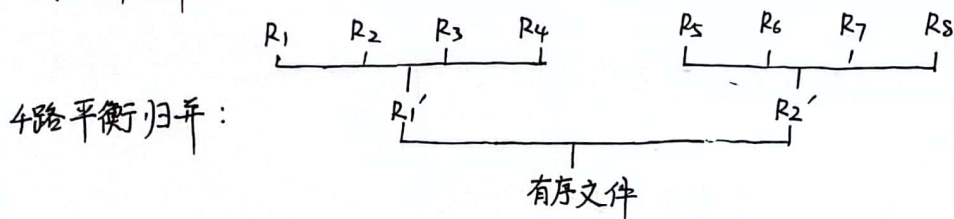
外部排序总时间 = 内部排序总时间 + 外存信息读/写时间 + 内部归并的时间



(2) K路平衡归并: 若初始归并段个数为 r ,

在内存中分配
K个输入缓冲区和
1个输出缓冲区

- ① 每趟将最多 K 个归并段归并为一个;
- ② 第一趟可归并为 $\lceil r/K \rceil$ 个归并段,
以后每趟将 m 个归并段归并成 $\lceil m/K \rceil$ 个归并段。

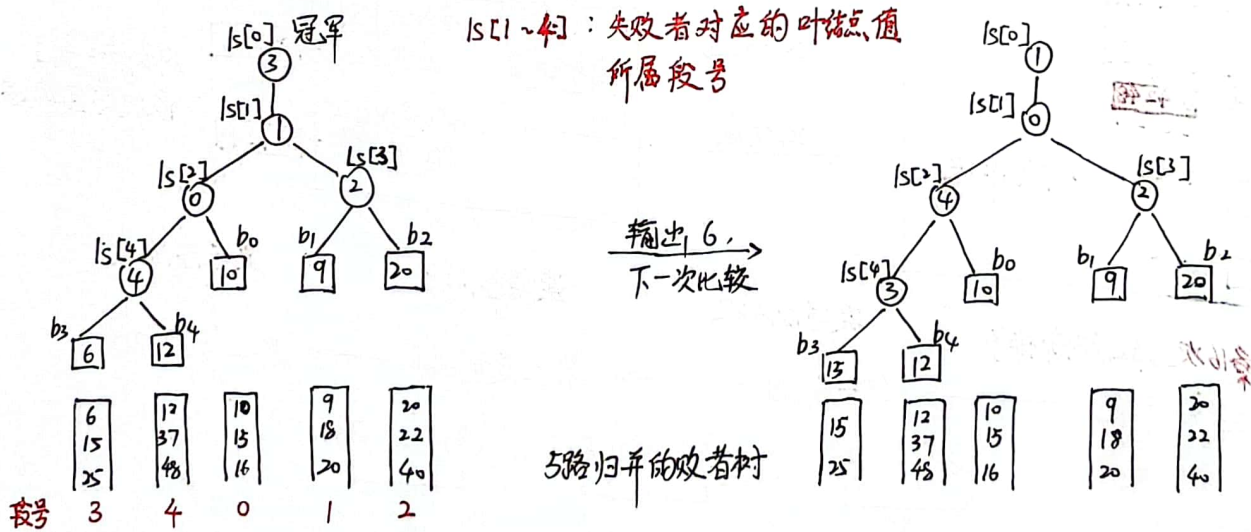


归并趟数 $S = \lceil \log_k r \rceil = \text{树高} - 1$

只要增大归并路数 k , 或减少初始归并段数 r , 都能减少归并趟数 S , 进而减少读/写磁盘次数, 达到提高外部排序速度的目的。

3. 败者树

- ① 基本概念: ① 完全二叉树。
- ② K 个叶结点分别对应 K 个归并段中当前参加比较的元素。
- ③ 非叶子结点用来记忆左右子树中的“失败者”。
- ④ 胜者往上比较, 直至根结点。



(2) 性能: ① K 路归并的败者树深度为 $\lceil \log_2 K \rceil + 1$, 故从 K 个记录中选择最小关键字, 仅需进行 $\lceil \log_2 K \rceil$ 次比较。

② 第一次构造败者树需要 $K-1$ 次比较。

③ 总比较次数约为: $S(n-1) \lceil \log_2 K \rceil = \lceil \log_k r \rceil (n-1) \lceil \log_2 K \rceil = \frac{(n-1) \lceil \log_2 r \rceil}{\log_2 K}$

$K \uparrow \rightarrow$ 输入缓冲区个数 $\uparrow$ 内存一定时, 缓冲区容量 $\downarrow$

$\downarrow$
内外存交换数据次数增大。

内部归并比较次数与 K 无关,
因此可适当增大 K ,
从而减少归并树高度,
减少 I/O 次数。

4. 置换——选择排序(生成初始归并段)

命题追踪 ▶▶ 置换-选择排序生成初始归并段的实例 (2023)

设初始待排文件为 FI，初始归并段输出文件为 FO，内存工作区为 WA，FO 和 WA 的初始状态为空，WA 可容纳 w 个记录。置换-选择算法的步骤如下：

- 1) 从 FI 输入 w 个记录到工作区 WA。
- 2) 从 WA 中选出其中关键字取最小值的记录，记为 MINIMAX 记录。
- 3) 将 MINIMAX 记录输出到 FO 中去。
- 4) 若 FI 不空，则从 FI 输入下一个记录到 WA 中。
- 5) 从 WA 中所有关键字比 MINIMAX 记录的关键字大的记录中选出最小关键字记录，作为新的 MINIMAX 记录。
- 6) 重复 3) ~5)，直至在 WA 中选不出新的 MINIMAX 记录为止，由此得到一个初始归并段，输出一个归并段的结束标志到 FO 中去。
- 7) 重复 2) ~6)，直至 WA 为空。由此得到全部初始归并段。

设待排文件 $FI = \{17, 21, 05, 44, 10, 12, 56, 32, 29\}$ ，WA 容量为 3，排序过程如表 8.2 所示。

表 8.2 置换-选择排序过程示例

输出文件 FO	工作区 WA	输入文件 FI
—	—	17, 21, 05, 44, 10, 12, 56, 32, 29
—	17 21 05	44, 10, 12, 56, 32, 29
05	17 21 44	10, 12, 56, 32, 29
05 17	10 21 44	12, 56, 32, 29
05 17 21	10 12 44	56, 32, 29
05 17 21 44	10 12 56	32, 29
05 17 21 44 56	10 12 32	29
05 17 21 44 56 #	10 12 32	29
10	29 12 32	—
10 12	29 32	—
10 12 29	32	—
10 12 29 32	—	—
10 12 29 32 #	—	—

上述算法，在 WA 中选择 MINIMAX 记录的过程需利用败者树来实现。

5. 最佳归并树

(1) 理论基础: ① 每个初始归并段对应一个叶子结点, 把归并段的块数作为叶子的权值.

② 归并树的 WPL = 树中所有叶结点的带权路径长度之和

③ 归并过程中磁盘的 I/O 次数 = 归并树的 WPL $\times 2$.

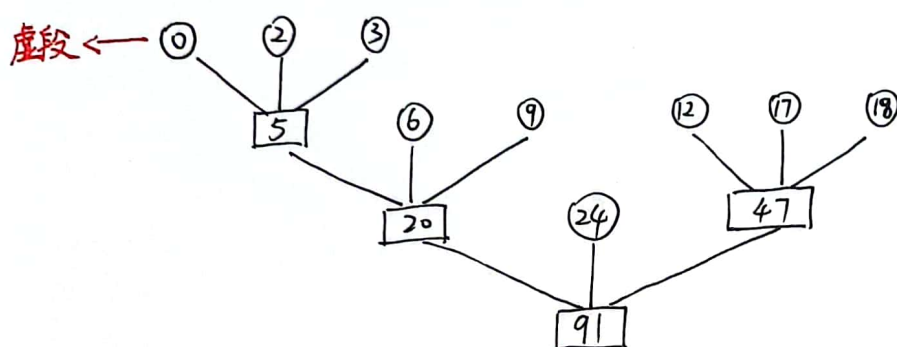
(2) 构造方法: ① 补充虚段: 若 $(\text{初始归并段数量} - 1) \% (k - 1) = 0$, 不需要添加.

若 $(\text{初始归并段数量} - 1) \% (k - 1) = u \neq 0$, 补充 $k - u - 1$ 个虚段.

② 构造 k 叉哈夫曼树: 每次选择 k 个根结点权值最小的树合并,

并将 k 个根结点的权值之和作为新的根结点的权值.

k 叉归并的最佳归并树一定是严格 k 叉树, 即树中只有度为 k, 度为 0 的结点.



$$\begin{aligned} WPL_{\min} &= (2+3+0) \times 3 + (6+9+12+17+18) \times 2 \\ &\quad + 24 \times 1 = 163 \\ I/O \text{ 次数} &= 163 \times 2 = 326 \text{ 次} \end{aligned}$$

8 个归并段的 3 叉最佳归并树

表 8.1 各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	否
2 路归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是